SOFTWARE REQUIREMENTS SPECIFICATION

LPA

LOGIC PUZZLE ARENA

A Browser-Based Multi-Game Brain-Training Platform

Five puzzles. One brain. One unified scoring engine.
A frontend web application built with HTML5, CSS3, Vanilla JavaScript (ES6+) and LocalStorage — designed as a college software engineering project demonstrating game logic, persistent client-side state, analytics and modular architecture.

Document Type	Version	Status	Date
IEEE 830-Style SRS	1.0	Approved for Development	August 14, 2026

Table of Contents

Logic Puzzle Arena — SRS v1.0 — Prepared for academic submission and faculty evaluation

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Logic Puzzle Arena**, a browser-based brain-training platform that combines five classic logic puzzles — Sudoku Lite, Memory Grid, Sequence Puzzle, Number Puzzle (sliding tile) and Word Scramble — into a single cohesive product with unified scoring, persistent statistics, achievements and a daily challenge. This document is intended to guide requirement analysis, UI/UX design, implementation, testing and faculty evaluation of the project, and to serve as the primary reference for project documentation and final presentation.

1.2 Document Conventions

Functional requirements are identified using the format `FR-<Module>-<Number>` (e.g., `FR-SUD-03`). Each requirement carries a priority of **High**, **Medium** or **Low**. This document follows the general structure of the IEEE 830 / IEEE 29148 SRS template, adapted to an appropriate scope for a college-level front-end software engineering project.

1.3 Intended Audience and Reading Suggestions

Audience	Recommended Focus
Faculty Evaluator / Project Guide	Sections 1, 2, 3, 5, 9, 10 — scope, requirements coverage, testing rigor and planning
Student Development Team	Sections 3, 4, 6, 8 — detailed requirements, interfaces, architecture and data model
UI/UX Designer	Sections 2, 3, 4.1, 6.1 — user flows, screens and interaction requirements
Tester / QA	Section 9 — test strategy, test categories and sample test cases

1.4 Project Scope

Logic Puzzle Arena is a **client-side only** web application (Phase 1) that runs entirely in the browser using HTML5, CSS3 and Vanilla JavaScript (ES6+), with **LocalStorage** as the persistence layer. The product delivers five playable puzzle games, a unified cross-game scoring formula, a player statistics dashboard, an achievement system and a date-based daily challenge — all without a server, database, authentication, or network dependency. Section 8 additionally documents a **forward-looking Phase 2 design** (REST API and relational database) purely for planning purposes, so the system can be evaluated on database/API design skills without requiring backend implementation in the current phase.

> **Explicitly out of scope for Phase 1:** user accounts/login, server-side storage, multiplayer/leaderboard sync across devices, payments, push notifications, and native mobile apps.

1.5 References & Definitions

Term	Definition
SRS	Software Requirements Specification
LocalStorage	Browser Web Storage API used to persist key–value data client-side across sessions
DOM	Document Object Model — the browser's in-memory representation of the HTML page
FR	Functional Requirement
NFR	Non-Functional Requirement
Streak	Number of consecutive days or consecutive correct puzzles completed by the player
XP	Experience Points awarded for completing puzzles and daily challenges

References: IEEE Std 830-1998 (Recommended Practice for SRS); MDN Web Docs — Web Storage API, HTML5, CSS Grid/Flexbox.

2. Overall Description

2.1 Product Perspective

Logic Puzzle Arena is a new, standalone, self-contained product. It is not a component of an existing system. Architecturally it is a static front-end web application: all pages, styling, game logic and data persistence run inside the user's browser, requiring only a static file host (or even a local file system) to operate — no backend server is required for Phase 1.

2.2 Product Features (Summary)

- Platform home page with player snapshot (best score, streak, puzzles solved) and puzzle selection cards
- Five independent puzzle engines: Sudoku Lite (6×6), Memory Grid, Sequence Puzzle, Number Puzzle (sliding tile), Word Scramble
- Three difficulty levels (Easy / Medium / Hard) selectable before each puzzle
- Unified scoring engine (base score, accuracy bonus, speed bonus, hint/mistake penalties)
- Player statistics dashboard with per-game best scores and accuracy
- Achievement system with unlock notifications
- Date-based daily challenge (no server required)
- Persistent state via LocalStorage (stats, settings, achievements, unfinished games)

2.3 User Classes and Characteristics

User Class	Description	Technical Skill
Casual Player	Plays puzzles for entertainment/brain-training; primary end user	None required
Returning Player	Comes back to beat previous scores, maintain streaks, chase achievements	None required
Faculty Evaluator	Reviews the deployed app and documentation for grading	General web usage
Developer/Maintainer	Student team member extending or debugging the codebase	HTML/CSS/JS proficiency

2.4 Operating Environment

Client	Any modern desktop or mobile browser (Chrome, Firefox, Edge, Safari — last 2 major versions)
Server	None required for Phase 1; any static file host (GitHub Pages, Netlify, Vercel, local server) is sufficient
Storage	Browser LocalStorage (≈5 MB quota, per-origin)
Screen Support	Responsive layout — desktop (≥1024px), tablet (≥768px), mobile (≥360px)

2.5 Design and Implementation Constraints

- Must be built using HTML5, CSS3 and Vanilla JavaScript ES6+ only — no frameworks (React/Vue/Angular) and no build tools required for Phase 1
- No backend, database, or authentication in Phase 1 — all persistence via LocalStorage
- Number Puzzle must generate boards via valid random moves from a solved state (never a raw shuffle) to guarantee solvability
- Sudoku is implemented as 6×6 ("Lite") rather than full 9×9 to keep validation logic manageable within project timelines
- Application must function fully offline once loaded (no external API dependency for core gameplay)

2.6 Assumptions and Dependencies

- The evaluating browser has LocalStorage enabled (not in private/incognito restricted mode) and JavaScript enabled
- A single browser profile represents a single player; no cross-device sync is expected in Phase 1
- Clearing browser data / cache will reset all player progress, which is an accepted limitation of Phase 1
- The project will be developed and evaluated by a small student team (2–4 members) within a single academic semester

3. System Features & Functional Requirements

This section details each functional module of Logic Puzzle Arena. Each module lists its purpose, priority and specific functional requirements (FR).

3.1 Home / Navigation Module

Description: Landing page that establishes the "platform" feel — greeting, player snapshot, today's challenge banner, and the five puzzle-selection cards.

ID	Requirement	Priority
FR-NAV-01	The system shall display a home page with the title, tagline and a "Start Training" call-to-action.	High
FR-NAV-02	The system shall display the player's current streak, best score and total puzzles solved, read from LocalStorage.	High
FR-NAV-03	The system shall render five game-selection cards (Sudoku, Memory, Sequence, Number, Scramble), each linking to its respective game page.	High
FR-NAV-04	The system shall allow the player to select a difficulty (Easy/Medium/Hard) before entering any puzzle.	High
FR-NAV-05	The system shall provide a persistent navigation link/button to the Player Dashboard from every page.	Medium

3.2 Sudoku Lite Module (6×6)

ID	Requirement	Priority
FR-SUD-01	The system shall generate a valid, solvable 6×6 Sudoku board and a corresponding puzzle board with a difficulty-dependent number of cells hidden.	High
FR-SUD-02	The system shall let the player enter a number into any empty cell using click/tap and keyboard input.	High
FR-SUD-03	The system shall validate each entry against the stored solution and visually highlight correct entries.	High
FR-SUD-04	The system shall flag incorrect entries with a visual (red) animation and increment a mistake counter.	High
FR-SUD-05	The system shall end the game when the mistake counter reaches three (3) and display a "Game Over" state.	High
FR-SUD-06	The system shall provide a "Hint" action that reveals one correct cell and applies a scoring penalty.	Medium
FR-SUD-07	The system shall display a running timer (mm:ss) throughout the game.	Medium
FR-SUD-08	Difficulty (Easy/Medium/Hard) shall control the number of pre-filled vs. hidden cells.	High

3.3 Memory Grid Module

ID	Requirement	Priority
FR-MEM-01	The system shall randomly highlight a subset of grid cells for approximately two seconds, then hide them.	High

FR-MEM-02	The system shall allow the player to click cells to reproduce the highlighted pattern.	**High**
FR-MEM-03	The system shall give immediate correct/incorrect feedback per click.	**High**
FR-MEM-04	The system shall advance to the next level with an increased number of highlighted cells upon successful pattern completion.	**High**
FR-MEM-05	The system shall end the round when the player selects an incorrect cell or exceeds the allotted attempts.	**Medium**

3.4 Sequence Puzzle Module

ID	Requirement	Priority
FR-SEQ-01	The system shall present a numeric sequence with a missing final term and four multiple-choice options.	**High**
FR-SEQ-02	Sequence questions shall be stored as predefined JavaScript objects (sequence, options, answer, difficulty) and selected at random per difficulty.	**High**
FR-SEQ-03	The system shall validate the selected option against the stored answer and display "Correct!" or "Incorrect" feedback with points awarded.	**High**
FR-SEQ-04	The system shall advance automatically to the next sequence question after feedback is shown.	**Medium**

3.5 Number Puzzle Module (Sliding Tile)

ID	Requirement	Priority
FR-NUM-01	The system shall generate the puzzle board by starting from a solved state and performing a large number of random *valid* moves, guaranteeing solvability.	**High**
FR-NUM-02	The system shall allow moving only a tile that is adjacent to the blank cell.	**High**
FR-NUM-03	The system shall track and display the move count and elapsed time.	**High**
FR-NUM-04	The system shall detect the win condition (tiles in ascending order with blank last) and display a completion state.	**High**
FR-NUM-05	Board size shall scale with difficulty: 3×3 (Easy), 4×4 (Medium), 5×5 (Hard).	**High**
FR-NUM-06	The system shall persist and display the player's best (lowest) move count per difficulty.	**Medium**

3.6 Word Scramble Module

ID	Requirement	Priority
FR-WRD-01	The system shall select a word from a predefined category list and display its letters in scrambled order.	**High**
FR-WRD-02	The system shall display an accompanying hint for the scrambled word.	**High**
FR-WRD-03	The system shall provide a text input and "Submit" action, validating the answer case-insensitively.	**High**
FR-WRD-04	The system shall track and display the player's current correct-answer streak.	**Medium**
FR-WRD-05	Word difficulty (easy/medium/hard) shall be tagged per word and filtered by the selected difficulty.	**Medium**

3.7 Unified Scoring Engine

Description: A single scoring module (`scoring.js`) consumed by all five games so results are directly comparable across the platform.

ID	Requirement	Priority
FR-SCR-01	The system shall compute a puzzle's score as: `Score = BaseScore + AccuracyBonus + TimeBonus − HintPenalty − MistakePenalty`.	High
FR-SCR-02	The system shall display a "Puzzle Complete" summary screen showing time, accuracy, hints used, difficulty and the itemized score breakdown.	High
FR-SCR-03	The system shall apply the same scoring formula consistently across all five games.	High
FR-SCR-04	The system shall update total score, streak and per-game best score in LocalStorage immediately after each puzzle.	High

3.8 Player Dashboard Module

ID	Requirement	Priority
FR-DSH-01	The system shall display aggregate statistics: total score, puzzles solved, overall accuracy, current streak and favourite (most-played) game.	High
FR-DSH-02	The system shall display a per-game breakdown table (best score and accuracy) for all five puzzles.	High
FR-DSH-03	The system shall recompute and reflect dashboard values immediately after every completed puzzle.	Medium

3.9 Achievement System

ID	Requirement	Priority
FR-ACH-01	The system shall define a fixed set of achievements (see Appendix A) with unlock conditions evaluated after each puzzle.	Medium
FR-ACH-02	The system shall display an animated notification when a new achievement is unlocked.	Medium
FR-ACH-03	The system shall persist unlocked achievements in LocalStorage and prevent duplicate unlock notifications.	Medium

3.10 Daily Challenge Module

ID	Requirement	Priority
FR-DLY-01	The system shall deterministically select a predefined daily challenge (game, difficulty, objective) based on the current calendar date, requiring no server.	Medium
FR-DLY-02	The system shall display the daily challenge and its XP reward on the home page.	Medium
FR-DLY-03	The system shall mark the daily challenge as completed for the current date once its objective is met, and shall not re-award it the same day.	Medium

3.11 Data Persistence Module

ID	Requirement	Priority
FR-STO-01	The system shall persist player statistics, high scores, settings, achievements and in-progress games in browser LocalStorage.	High
	The system shall gracefully handle a missing or corrupted LocalStorage entry by initializing default values.	Medium

FR-STO-02		
FR-STO-03	The system shall provide a "Reset Progress" option in settings that clears all stored player data after confirmation.	**Low**

4. External Interface Requirements

4.1 User Interfaces

- Card-based home page with clear visual hierarchy: hero banner → today's challenge → game cards
- Consistent puzzle-arena layout across all five games: board/prompt area, stats bar (timer/moves/mistakes), and action buttons
- A standard "Puzzle Complete" results modal/page shared by all games, driven by the scoring engine
- Dashboard page with summary cards and a per-game statistics table
- Responsive design supporting desktop, tablet and mobile breakpoints using CSS Grid/Flexbox
- Toast-style animated notifications for achievement unlocks

4.2 Hardware Interfaces

No special hardware is required. The system uses standard input devices (mouse, touchscreen, keyboard) available on any consumer desktop, laptop, tablet or smartphone.

4.3 Software Interfaces

Interface	Purpose
Web Browser (Chrome/Firefox/Edge/Safari)	Renders HTML/CSS, executes JavaScript, hosts the LocalStorage API
Web Storage API (LocalStorage)	Client-side key–value persistence for all player data
Static File Host (optional)	Serves static HTML/CSS/JS assets (e.g., GitHub Pages) for deployment/demo

4.4 Communication Interfaces

Phase 1 requires no network communication for core functionality — all game logic and data access are local to the browser. Phase 2 (Section 8) introduces HTTPS-based REST API communication for optional cloud sync.

5. Non-Functional Requirements

5.1 Performance

ID	Requirement
NFR-PRF-01	Any page shall load and become interactive within 2 seconds on a typical broadband/local connection.
NFR-PRF-02	Puzzle-board generation (Sudoku, Number Puzzle) shall complete within 200ms so as not to block the UI.
NFR-PRF-03	Score computation and dashboard updates shall be reflected within 100ms of puzzle completion.

5.2 Usability

ID	Requirement
NFR-USE-01	A first-time user shall be able to start their first puzzle within 3 clicks from the home page.
NFR-USE-02	All interactive elements shall provide visible hover/focus/active states.
NFR-USE-03	Color choices for correct/incorrect feedback shall not rely on color alone (icon/animation shall accompany color).

5.3 Reliability & Availability

ID	Requirement
NFR-REL-01	The application shall not lose previously saved statistics due to a normal browser refresh or navigation.
NFR-REL-02	The application shall remain fully usable while offline, once initially loaded.

5.4 Security

ID	Requirement
NFR-SEC-01	All user-entered text (e.g., Word Scramble answers) shall be sanitized before being rendered back into the DOM to prevent injection.
NFR-SEC-02	No personally identifiable information shall be collected or stored in Phase 1.

5.5 Maintainability

ID	Requirement
NFR-MNT-01	Each puzzle's logic shall reside in its own JavaScript module/file, isolated from other games.
NFR-MNT-02	Shared concerns (scoring, storage) shall be implemented once in `scoring.js` and `storage.js` and reused by all games.

5.6 Portability

ID	Requirement
NFR-PRT-01	The application shall run without modification on Windows, macOS, Linux, Android and iOS via any evergreen browser.

5.7 Scalability (Forward-looking)

ID	Requirement
NFR-SCL-01	The data and module structure shall allow additional puzzle games to be added without redesigning the scoring or storage layers.

6. System Architecture & Design

6.1 High-Level Architecture Overview

Presentation Layer
HTML5 pages: index.html, dashboard.html, games/*.html — structure, forms, boards, modals

Styling Layer
CSS3: global.css, dashboard.css, games.css — Grid/Flexbox layout, themes, animations

Logic Layer
Vanilla JS ES6+ modules: main.js, scoring.js, storage.js, and one file per game engine

Game Engines
sudoku.js · memory.js · sequence.js · number.js · scramble.js — independent, isolated per game

Shared Services
scoring.js (unified formula) · storage.js (LocalStorage read/write, defaults, reset)

Persistence Layer
Browser LocalStorage — player profile, per-game stats, achievements, daily challenge state

6.2 Data Model (LocalStorage Schema)

All player data is stored under a small set of namespaced LocalStorage keys, each holding a JSON string.

```
localStorage: "lpa_playerProfile"
{
  totalScore: 8420,
  puzzlesSolved: 27,
  overallAccuracy: 87,
  currentStreak: 5,
  favouriteGame: "memory",
  lastPlayedDate: "2026-08-13"
}

localStorage: "lpa_gameStats"
{
  sudoku:   { best: 820,  accuracy: 89, attempts: 6  },
  memory:   { best: 1240, accuracy: 94, attempts: 9  },
  sequence: { best: 960,  accuracy: 82, attempts: 5  },
  number:   { best: 740,  accuracy: 86, attempts: 4  },
  scramble: { best: 1110, accuracy: 91, attempts: 3  }
}

localStorage: "lpa_achievements"
{ firstBlood: true, speedDemon: true, perfectMind: false,
  unstoppable: false, puzzleMaster: false }

localStorage: "lpa_dailyChallenge"
{ date: "2026-08-14", game: "memory", difficulty: "hard",
  objective: "Complete Level 8 without a mistake",
  rewardXP: 500, completed: false }

localStorage: "lpa_settings"
{ soundEnabled: true, theme: "green", difficultyDefault: "medium" }
```

6.3 Project Directory Structure

```
logic-puzzle-arena/
├── index.html                → Home page
├── dashboard.html             → Player statistics dashboard
├── games/
|   ├── sudoku.html   memory.html   sequence.html
|   ├── number.html   scramble.html
├── css/
```

```
│   ├── global.css   dashboard.css   games.css
├── js/
│   ├── main.js        → navigation, difficulty selection
│   ├── storage.js      → LocalStorage read/write/reset
│   ├── scoring.js       → unified scoring formula
│   ├── sudoku.js  memory.js  sequence.js  number.js  scramble.js
└── assets/
    ├── icons/   images/   sounds/
```

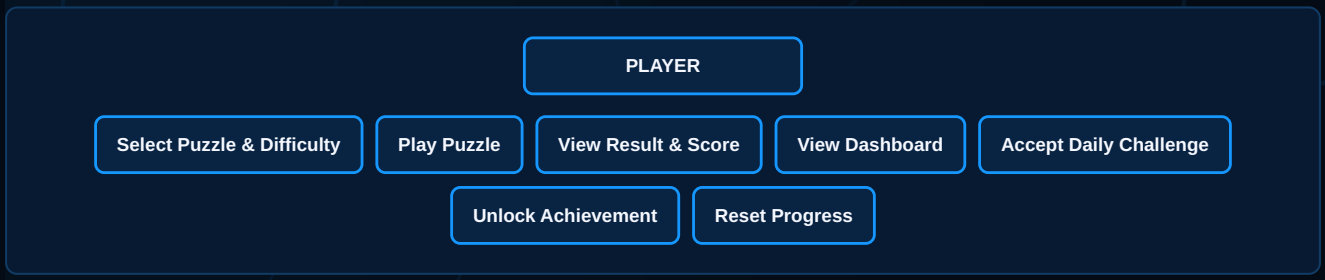
6.4 Application Flow / State Diagram

7. Use Case Model

7.1 Actors

- **Player** — the primary actor who plays puzzles, views stats and earns achievements
- **System (Browser/LocalStorage)** — secondary actor that persists and returns player data

7.2 Use Case Summary Diagram



7.3 Use Case Descriptions

Use Case	Primary Actor	Preconditions	Main Success Scenario
UC-01 Play a Puzzle	Player	Home page loaded	Player selects game and difficulty → plays → system scores and saves result → result screen shown
UC-02 View Dashboard	Player	At least one puzzle completed (else defaults shown)	Player opens Dashboard → system reads LocalStorage → aggregate and per-game stats rendered
UC-03 Accept Daily Challenge	Player	Current date's challenge not yet completed	Player views challenge on home page → plays the specified game/ difficulty/objective → system verifies objective → XP awarded
UC-04 Unlock Achievement	Player	A qualifying event occurs (e.g., first puzzle solved)	System evaluates achievement conditions after each puzzle → shows unlock notification → persists new state
UC-05 Reset Progress	Player	Player opens Settings	Player selects "Reset Progress" → confirms → system clears all LocalStorage keys → defaults re-initialized

8. Future Phase Planning — Backend, API & Database

Phase 1 requires no backend. This section documents an optional, forward-compatible **Phase 2** design so the project can additionally demonstrate database and API planning skills during evaluation, without adding implementation risk to the core deliverable.

8.1 Proposed Backend Architecture (Phase 2)

Client
Existing HTML/CSS/JS front end, extended with an optional "Sign in to sync" flow

API Server
Node.js + Express (or similar) exposing a REST API over HTTPS

Database
Relational database (e.g., PostgreSQL/ MySQL) for durable, cross-device player data

8.2 Proposed REST API Endpoints

Method & Endpoint	Purpose
POST /api/auth/register	Create a new player account
POST /api/auth/login	Authenticate a player and issue a session token
GET /api/players/{id}/profile	Fetch aggregate player profile (score, streak, accuracy)
GET /api/players/{id}/stats	Fetch per-game statistics
POST /api/results	Submit a completed puzzle result for scoring and persistence
GET /api/achievements/{id}	Fetch unlocked achievements for a player
GET /api/daily-challenge	Fetch today's server-defined daily challenge
GET /api/leaderboard	Fetch top players by total score (future feature)

8.3 Proposed Database Schema (ER Outline)

Table	Key Columns
players	player_id (PK), username, email, password_hash, created_at
game_results	result_id (PK), player_id (FK), game_type, difficulty, score, accuracy, time_taken, hints_used, played_at
achievements	achievement_id (PK), name, description, unlock_condition
player_achievements	player_id (FK), achievement_id (FK), unlocked_at
daily_challenges	challenge_id (PK), date, game_type, difficulty, objective, reward_xp

Note: This section is a design exercise for future extensibility and evaluation of database/API planning skills. It is not required for Phase 1 delivery or grading of core functionality.

9. Software Testing Plan

9.1 Testing Approach

Testing will be performed manually and via lightweight scripted checks, appropriate to a front-end-only student project. The strategy combines **unit-level** testing of pure logic functions (scoring formula, board generators, solvability checks), **functional/UI** testing of each game against its FRs, and **exploratory usability** testing across desktop and mobile viewports.

9.2 Test Case Categories

Category	Focus
Unit Testing	Scoring formula correctness, Sudoku solution validator, Number Puzzle solvability generator, Word Scramble letter-shuffle/ validate logic
Functional Testing	Each FR in Section 3 verified against its expected behavior
Integration Testing	Game result correctly updates storage.js and reflects on Dashboard and Achievements
UI/Responsive Testing	Layout correctness at desktop, tablet and mobile breakpoints
Persistence Testing	Data survives page refresh; corrupted/missing keys fall back to defaults
Regression Testing	Re-run core test cases after each new feature is merged

9.3 Sample Test Cases

TC ID	Description	Expected Result
TC-01	Generate a Number Puzzle board on Medium (4×4)	Board is solvable; blank tile present; exactly one blank
TC-02	Enter 3 incorrect Sudoku values	Game Over state triggers after the 3rd mistake
TC-03	Complete Memory Grid Level 1 correctly	Player advances to Level 2 with more highlighted cells
TC-04	Submit correct Word Scramble answer in different letter case	Answer accepted (case-insensitive match)
TC-05	Complete any puzzle with 1 hint used and 90%+ accuracy	Score = Base + AccuracyBonus + TimeBonus − HintPenalty, matching formula in FR-SCR-01
TC-06	Refresh the browser after completing a puzzle	Dashboard still reflects the updated stats (LocalStorage persisted)
TC-07	Open the app with an empty LocalStorage (first-time user)	Default profile values (0 score, 0 streak) render without errors
TC-08	Reach a 10-puzzle streak	"UNSTOPPABLE" achievement unlocks with notification
TC-09	Load home page on a 375px-wide mobile viewport	Game cards reflow to a single column; no horizontal scroll
TC-10	Complete the Daily Challenge objective	XP is awarded once; re-attempting the same day does not re-award XP

10. Project Plan & Development Roadmap

10.1 Recommended Build Order

To manage risk and demonstrate incremental progress, the team should implement games in increasing order of logic complexity: **Memory Grid → Word Scramble → Sequence Puzzle → Number Puzzle → Sudoku Lite.** Memory Grid and Word Scramble validate the shared scoring/storage pipeline quickly; Sudoku, the most logic-heavy module, is built last once the platform shell is proven.

10.2 Work Breakdown Structure

#	Work Package	Deliverable
1	Shared navigation & home UI	index.html, global.css, main.js
2	Five puzzle mechanics	games/*.html + js/*.js per game
3	Reusable timer, score & difficulty system	scoring.js
4	Integrate all games with scoring engine	Result screens wired to scoring.js
5	LocalStorage persistence layer	storage.js
6	Statistics / Dashboard page	dashboard.html, dashboard.css
7	Polish: animations, sound, achievements, responsive layout	games.css, assets/, achievement notifications

10.3 Suggested Milestone Timeline (12-Week Semester)

Week(s)	Milestone
1–2	SRS finalization, wireframes, project setup, home page shell
3–4	Memory Grid + Word Scramble complete and playable
5–6	Sequence Puzzle + Number Puzzle complete and playable
7–8	Sudoku Lite complete; scoring engine integrated across all games
9	LocalStorage persistence finalized; Dashboard page built
10	Achievements + Daily Challenge implemented
11	Responsive polish, animations, cross-browser testing (Section 9)
12	Final documentation, presentation deck, demo rehearsal, submission

11. Appendices

Appendix A — Achievement Catalogue

Achievement	Unlock Condition
First Blood	Solve your first puzzle
Speed Demon	Complete a puzzle in under 30 seconds
Perfect Mind	Finish a puzzle with 100% accuracy
Unstoppable	Reach a 10-puzzle completion streak
Puzzle Master	Complete every one of the five puzzles on Hard difficulty

Appendix B — Glossary

Base Score	Fixed points awarded for completing a puzzle, before bonuses/penalties
Accuracy Bonus	Points added based on correct-to-total attempt ratio
Speed/Time Bonus	Points added for completing a puzzle faster than a difficulty-based threshold
Hint Penalty	Points deducted per hint used
Mistake Penalty	Points deducted per incorrect attempt

Appendix C — Assumptions Summary

- Single-player, single-browser-profile experience (no accounts in Phase 1)
- Grading/demo will occur on a standard evergreen browser with LocalStorage available
- Team size of 2–4 student developers over one semester

Appendix D — Sample Requirements Traceability Matrix

Requirement	Design Element	Test Case
FR-SUD-05 (3-mistake game over)	sudoku.js — mistake counter logic	TC-02
FR-NUM-01 (guaranteed solvable board)	number.js — valid-move scrambler	TC-01
FR-SCR-01 (scoring formula)	scoring.js	TC-05
FR-STO-01 (LocalStorage persistence)	storage.js	TC-06, TC-07
FR-ACH-02 (unlock notification)	main.js achievement handler	TC-08

End of Document — Logic Puzzle Arena SRS v1.0